UNIVERSIDAD MARIANO GÁLVEZ DE GUATEMALA — INFORME FINAL

Ingeniería en Sistemas · Algoritmos (2026) · Proyecto 12: Transporte y Rutas (Variante A) · Catedrático: Ing. Mag. Jhensser Aguirre

Estudiante: [NOMBRE COMPLETO] | **Carné:** [CARNÉ — número par → Variante A] | **Estándar:** C++17 en CLion (CMake)

---

1. Descripción del Problema y Arquitectura Modular en C++17

El sistema administra los recursos de transporte de una terminal de autobuses mediante el control estricto de capacidades, recorridos, asignaciones de boletos e ingresos. Para cumplir con las buenas prácticas de Ingeniería en Sistemas sin utilizar librerías externas ni motores fuera del alcance del curso, el proyecto se estructuró en tres capas dentro de carpetas independientes en C++17:

- **Carpeta `frontend/` (Presentación y Validación):** Contiene `interfaz.h/.cpp` y `validaciones.h/.cpp`. Gestiona la interacción con el usuario en consola, el formato tabular y la validación de entradas mediante `getline(cin, linea)` + `istringstream`.
- **Carpeta `backend/` (Lógica de Negocio y Algoritmos):** Contiene `modelos.h`, `logica_rutas.h/.cpp` y `algoritmos.h/.cpp`. Implementa el registro de rutas, control de estados, venta/cancelación segura, el reto algorítmico de recomendación, búsquedas por 3 criterios y ordenamiento Burbuja manual.
- **Carpeta `database/` (Conexión a Base de Datos y Cifrado para Tontos):** Contiene `conexion_db.h/.cpp` y `cifrado_tontos.h/.cpp`. Administra la persistencia en `rutas.txt` y genera una copia cifrada en `rutas_cifradas.txt` aplicando un desplazamiento ASCII de +3 caracteres.

2. Requerimientos Funcionales y Trazabilidad Técnica

Cód.	Requerimiento Funcional	Archivo C++ Responsable	Función y Pruebas Asociadas
RF01	Registrar ruta validando ID único, rangos y origen ≠ destino.	`backend/logica_rutas.cpp`	`registrarRuta()` (P01–P08, P36)
RF02	Listar rutas con capacidad, vendidos, disponibles, precio e ingresos.	`backend/logica_rutas.cpp`	`listarRutas()` (P30, P33)

Cód.	Requerimiento Funcional	Archivo C++ Responsable	Función y Pruebas Asociadas
RF03	Activar/desactivar ruta conservando su historial en memoria y archivo.	`backend/logica_rutas.cpp`	`cambiarEstadoRuta()` (P14, P37)
RF04	Vender boletos por ID impidiendo ventas en rutas llenas o inactivas.	`backend/logica_rutas.cpp`	`venderBoletos()`, `aplicarVenta()` (P09–P14)
RF05	Cancelar boletos vendidos restando pasajeros y recalculando ingresos.	`backend/logica_rutas.cpp`	`cancelarBoletos()` (P15, P16)
RF06	Reto Algorítmico: recomendar ruta activa más barata con cupo para el grupo.	`backend/logica_rutas.cpp`	`recomendarYVender()` (P26–P29)
RF07–08	Búsqueda secuencial por ID, ciudad de origen y ciudad de destino.	`backend/algoritmos.cpp`	`buscarPorID/Origen/Destino()` (P17–P20)
RF09–10	Ordenamiento Burbuja manual por ingresos y generación de 3 reportes.	`backend/algoritmos.cpp`	`ordenarPorIngresos()`, `reporte*()` (P21–P25)
RF11–14	Conexión a Base de Datos con `fstream` y módulo **Cifrado para Tontos**.	`database/conexion_db.cpp`, `cifrado_tontos.cpp`	`guardarDatos()`, `cifrarParaTontos()` (P30–P33, P39–P42)

3. Diagramas de Arquitectura, Flujo y Pseudocódigo

A continuación se presentan los diagramas actualizados de la versión final del sistema, reflejando la división modular en tres capas (`frontend/`, `backend/` y `database/`), el flujo de lectura/escritura con `fstream` aplicando el algoritmo **Cifrado para Tontos** (+3 / -3 en la tabla ASCII) y el proceso de selección óptima del recurso de transporte.

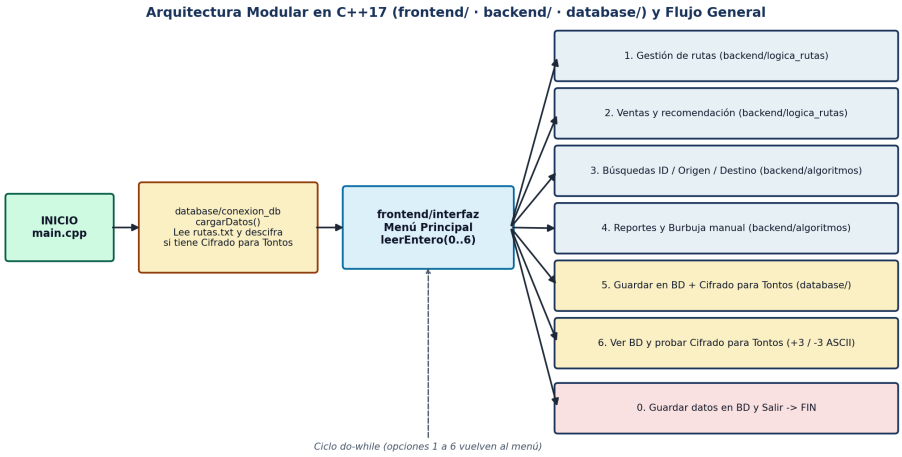


Figura 1: Arquitectura modular en C++17 (frontend, backend, database) y flujo general del sistema.

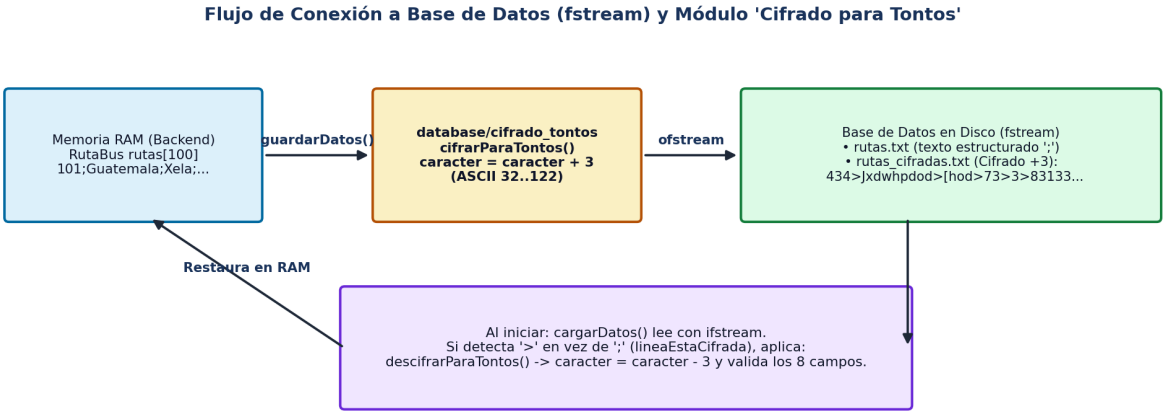


Figura 2: Conexión a Base de Datos con fstream y flujo del algoritmo "Cifrado para Tontos".

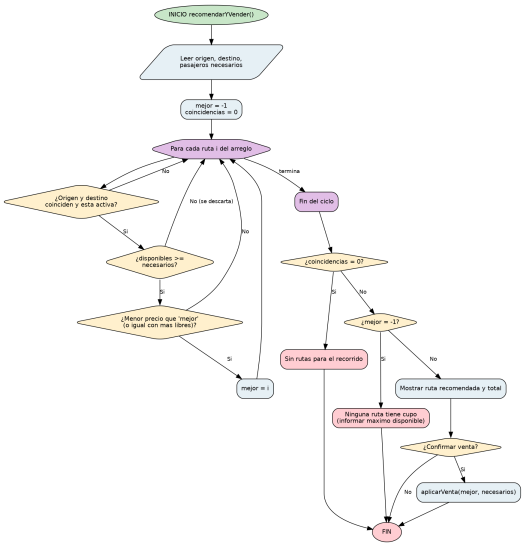


Figura 3: Diagrama de flujo del Reto Algorítmico (selección óptima de ruta por capacidad y tarifa).

4. Pseudocódigo de los Algoritmos Principales

```
// 1. Módulo "Cifrado para Tontos" (database/cifrado_tontos.cpp)
CONSTANTE CLAVE_TONTOS = 3
```

```
FUNCION cifrarParaTontos(textoOriginal)
    resultado <- textoOriginal
    PARA i DESDE 0 HASTA longitud(resultado) - 1 HACER
        SI resultado[i] >= 32 Y resultado[i] <= 122 ENTONCES
            resultado[i] <- resultado[i] + CLAVE_TONTOS
    RETORNAR resultado

FUNCION descifrarParaTontos(textoCifrado)
    resultado <- textoCifrado
    PARA i DESDE 0 HASTA longitud(resultado) - 1 HACER
        SI resultado[i] >= 35 Y resultado[i] <= 125 ENTONCES
            resultado[i] <- resultado[i] - CLAVE_TONTOS
    RETORNAR resultado

// 2. Reto Algorítmico: Recomendar ruta óptima (backend/logica_rutas.cpp)
FUNCION recomendarYVender(rutas, cantidad)
    LEER origen, destino, necesarios
    mejor <- -1; coincidencias <- 0; maxDisponibles <- 0
    PARA i DESDE 0 HASTA cantidad - 1 HACER
        SI mismoTexto(rutas[i].origen, origen) Y mismoTexto(rutas[i].destino, destino) Y
rutas[i].activa ENTONCES
            coincidencias <- coincidencias + 1
            disp <- rutas[i].capacidad - rutas[i].pasajeros
            SI disp > maxDisponibles ENTONCES maxDisponibles <- disp
            SI disp >= necesarios ENTONCES
                SI mejor = -1 O rutas[i].precio < rutas[mejor].precio O
                   (rutas[i].precio = rutas[mejor].precio Y disp > libres(mejor))
ENTONCES
                    mejor <- i
    SI mejor <> -1 Y usuarioConfirma ENTONCES aplicarVenta(rutas[mejor], necesarios)
```

5. Evidencias de Ejecución y Pruebas Automatizadas (42/42)

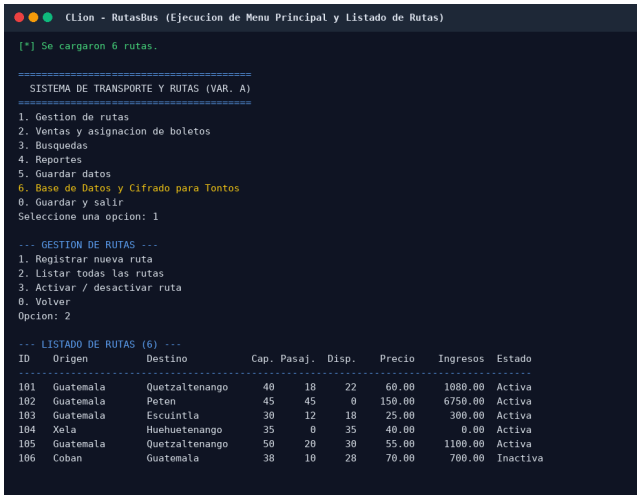


Figura 4: Ejecución en consola del menú principal y listado de rutas activas e inactivas.

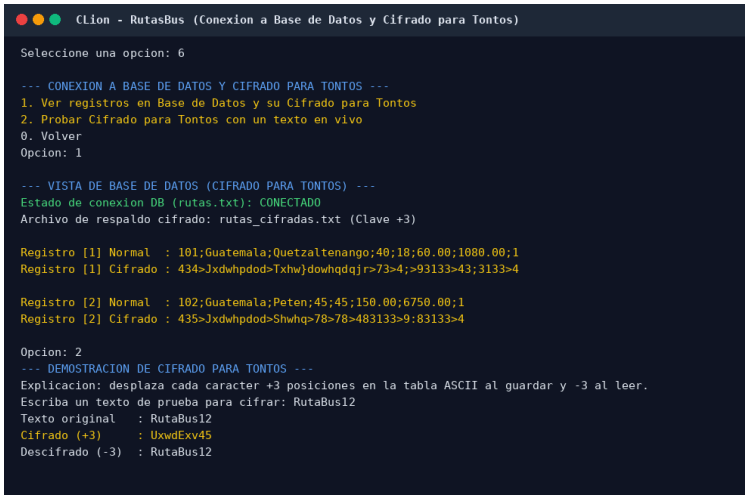


Figura 5: Vista de la conexión a Base de Datos y demostración en vivo del Cifrado para Tontos.

6. Manual Breve de Uso y Conclusiones

- **Compilación en CLion:** Abrir la carpeta `03_CODIGO_FINAL` desde *File > Open*. CLion configura automáticamente el proyecto usando `CMakeLists.txt` con C++17 e incluye las carpetas `frontend/`, `backend/` y `database/`.
- **Configuración de directorio de datos:** En *Run > Edit Configurations*, fijar `Working directory = $ProjectFileDir$` para que `rutas.txt` y `rutas_cifradas.txt` se almacenen en la carpeta del proyecto.
- **Conclusión:** El diseño modular en tres capas separa limpiamente la interfaz de consola, la lógica algorítmica y la persistencia en archivos, permitiendo defender cada función y realizar modificaciones en vivo en pocos segundos.